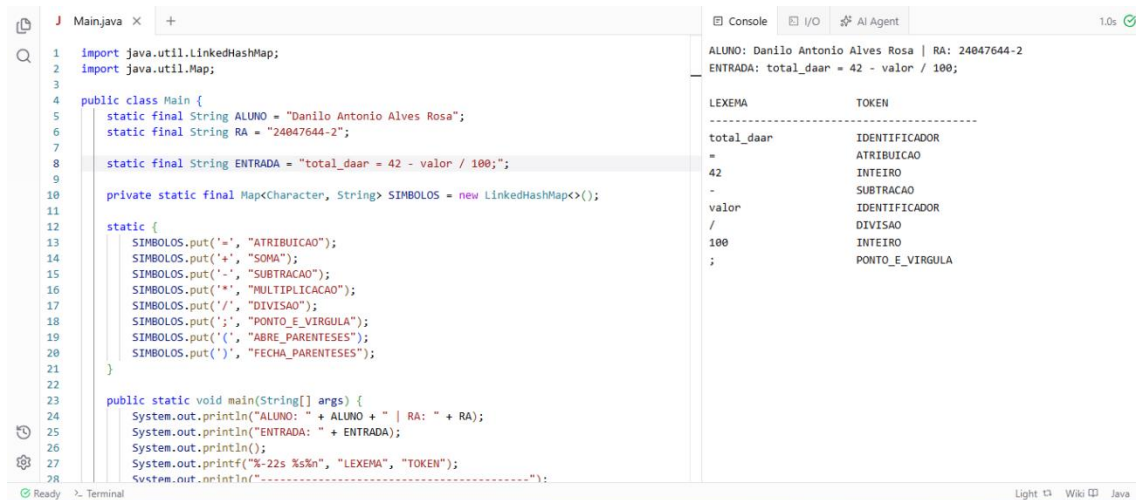


1: Tokens.png mostra a entrada personalizada e a lista de tokens.



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a class 'Main' with static variables for student name, RA, and a custom input string. It also defines a set of symbols and a main method that prints the input and its tokens.

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "Danilo Antonio Alves Rosa";
6     static final String RA = "24047644-2";
7
8     static final String ENTRADA = "total_daar = 42 - valor / 100;";
9
10    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12    static {
13        SIMBOLOS.put('=', "ATRIBUICAO");
14        SIMBOLOS.put('+', "SOMA");
15        SIMBOLOS.put('-', "SUBTRACAO");
16        SIMBOLOS.put('*', "MULTIPLICACAO");
17        SIMBOLOS.put('/', "DIVISAO");
18        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
19        SIMBOLOS.put('(', "ABRE_PARENTESES");
20        SIMBOLOS.put(')', "FECHA_PARENTESES");
21    }
22
23    public static void main(String[] args) {
24        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
25        System.out.println("ENTRADA: " + ENTRADA);
26        System.out.println();
27        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");
28        System.out.println("-----");
```

The console output shows the student information and the tokenization of the custom input string:

```
ALUNO: Danilo Antonio Alves Rosa | RA: 24047644-2
ENTRADA: total_daar = 42 - valor / 100;

LEXEMA      TOKEN
-----
total_daar  IDENTIFICADOR
=           ATRIBUICAO
42          INTEIRO
-           SUBTRACAO
valor       IDENTIFICADOR
/           DIVISAO
100         INTEIRO
;           PONTO_E_VIRGULA
```

2. Lexico.png mostra palavra reservada, identificador e erro léxico.



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a class 'Main' with static variables for student name, RA, and a custom input string. It also defines a set of reserved words and a main method that prints the input and its tokens.

```
1 import java.util.Arrays;
2 import java.util.HashSet;
3 import java.util.Set;
4
5 public class Main {
6     static final String ALUNO = "Danilo Antonio Alves Rosa";
7     static final String RA = "24047644-2";
8
9     static final String IDENTIFICADOR_PERSONALIZADO = "nota_daar";
10
11    static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>() {
12        {
13            Arrays.asList("int", "if", "else", "while", "return")
14        }
15    };
16
17    public static void main(String[] args) {
18        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
19        executarTeste("TESTE A - entrada compacta",
20            "int total=valor+10;");
21        executarTeste("TESTE B - espaços e comentário",
22            "int total = valor + 10; // este comentario sera ignorado");
23        executarTeste("TESTE C - reservadas e identificadores",
24            "int inteiro = 0; while (inteiro < intValue) inteiro = inteiro + 1;");
25        executarTeste("TESTE D - identificador personalizado",
26            "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
27        executarTeste("TESTE E - caractere inválido",
28            "int valor# = 1;");
29    }
30}
```

The console output shows the student information and the lexical analysis of the custom input string:

```
ALUNO: Danilo Antonio Alves Rosa | RA: 24047644-2

=== TESTE A - entrada compacta ===
ENTRADA: int total=valor+10;
LEXEMA      CLASSE      TOKEN
-----
int          LETRA       PALAVRA_RESERVADA
total        LETRA       IDENTIFICADOR
=            SIMBOLO     ATRIBUICAO
valor        LETRA       IDENTIFICADOR
+            SIMBOLO     SOMA
10           DIGITO      INTEIRO
;            SIMBOLO     PONTO_E_VIRGULA

=== TESTE B - espaços e comentário ===
ENTRADA: int total = valor + 10; // este comentario sera ignorado
LEXEMA      CLASSE      TOKEN
-----
int          LETRA       PALAVRA_RESERVADA
total        LETRA       IDENTIFICADOR
=            SIMBOLO     ATRIBUICAO
valor        LETRA       IDENTIFICADOR
+            SIMBOLO     SOMA
10           DIGITO      INTEIRO
```

3. _erro_compilador.png mostra uma tentativa que falhou.



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a class 'Main' with static variables for student name, RA, and a custom input string. It also defines a set of reserved words and a main method that prints the input and its tokens.

```
1 public class Main {
2     static final String ALUNO = "Danilo Antonio Alves Rosa";
3     static final String RA = "24047644-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15}
16
```

The console output shows the compilation errors:

```
Main.java:8: error: ';' expected
    int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
    ^
Main.java:10: error: ')' expected
    if (total > 20 { // CORRIGIR 2: feche corretamente o
    ^
Main.java:11: error: unclosed string literal
    System.out.println("Programa corrigido!"); // CORRIGIR
    ^
3 errors
Debug this error
```

4. _programa_corrigido.png mostra a execução depois dos reparos.



```
1 public class Main {
2     static final String ALUNO = "Danilo Antonio Alves Rosa";
3     static final String RA = "24847644-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e vírgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16 }
17 }
```

Console: ALUNO: Danilo Antonio Alves Rosa | RA: 24847644-2
Programa corrigido!
Resultado: 30

5. _descendente.png mostra expr, term ou factor no rastreamento.



```
1 public class Main {
2     static final String ALUNO = "Danilo Antonio Alves Rosa";
3     static final String RA = "24847644-2";
4
5     static final String ENTRADA = "(2 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecurcaoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29 }
```

Console: SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: \$FIM\$
SAINDO de <factor> | lookahead = \$FIM\$
SAINDO de <term> | lookahead = \$FIM\$
SAINDO de <expr> | lookahead = \$FIM\$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 20